

MANDAR JOSHI

Phone: +91 70202 12202 | Email: mandarjoshi1045@gmail.com

LinkedIn: linkedin.com/in/mandar-joshi-0b951b28a | GitHub: github.com/mandar1045

Portfolio: mandarjoshi.me

PROFESSIONAL SUMMARY

Software Engineer and active Open Source Contributor specializing in distributed systems, backend infrastructure, and API reliability. Core contributor to massive open-source ecosystems including **Supabase** (securing self-hosted Realtime infrastructure), **FOSSology** (resolving critical C/PHP memory bugs), and **Kubernetes** (CNCF). Architected production-grade microservice platforms in Go and TypeScript handling enterprise workflows, driven by a deep passion for building secure, scalable software in the open.

EDUCATION

Vellore Institute of Technology

Vellore, India

Bachelor of Technology in Information Technology

Jul. 2024 – May 2028 (Expected)

Relevant Coursework:

Data Structures & Algorithms, Database Management Systems, Operating Systems, Computer Networks, Object-Oriented Programming, Software Engineering, Machine Learning, Cloud Computing, Discrete Mathematics, Linear Algebra

TECHNICAL SKILLS

Programming Languages: Go, TypeScript, C, PHP, Elixir, Python, SQL, Bash, JavaScript

Backend & Systems: gRPC/Protobuf, Node.js, Express.js, FastAPI, REST APIs

Infrastructure & DevOps: Docker, Kubernetes, Linux, Nginx, Caddy, Terraform, GitHub Actions, CI/CD

Databases: PostgreSQL, Redis, MongoDB, Supabase; Schema Design, Query Optimization

Messaging & Streaming: Apache Kafka (Redpanda), Redis Pub/Sub, Event-Driven Architecture

Cloud Platforms: AWS (EC2, S3, ECS Fargate), Oracle Cloud

Developer Tools & Security: Git/GitHub, Postman, JWT Authentication, HMAC-SHA256, AES-256-GCM

Concepts: Microservices Architecture, Distributed Systems, High Availability, Idempotency, Rate Limiting

WORK EXPERIENCE & OPEN SOURCE

Open Source Contributor | Supabase Inc.

8 Merged PRs, 9 Open PRs, 5 Issues

GitHub: github.com/supabase/supabase

Key Contributions & Pull Requests:

- [supabase/realtime#2231: Fix WAL sender test flakes](#) — Fixed `max_wal_senders` test flakes caused by missing `public.test` table in Realtime.
- [supabase/realtime#2228: Consolidate extension tests](#) — Refactored and consolidated extension tests into one canonical location mirroring `lib/`.
- [supabase/realtime#2222: Suppress Ranch connection logs](#) — Added tests to verify that Ranch killed connection logs are properly suppressed in the tracker.
- [supabase/realtime#2174: Fix emoji slicing in fuzzy search](#) — Prevented emoji surrogate pair slicing bugs in the Realtime inspector's fuzzy search feature.
- [supabase/realtime#2167: Fix Ranch charlist log filter](#) — Fixed character list handling in the Ranch log filter to prevent logging errors.
- [supabase/supabase#50306: Proxy .well-known auth route](#) — Proxied `/.well-known/*` and `/pg/*` routes to the API gateway in Caddy/Nginx configs to fix 404s.
- [supabase/supabase#46021: Secure Realtime encryption key](#) — Moved hardcoded Realtime `DB_ENC_KEY` to a configurable `.env` variable for self-hosted instances.
- [supabase/server#160: Validate JWT audience & issuer](#) — Added strict validation for JWT audience and issuer fields to improve authentication security.

GitHub: github.com/fossology/fossology

Key Contributions & Pull Requests:

- [fossology/fossology#3605: Correct typos in user-facing messages](#) — Fixed typos across multiple UI components and user-facing CLI messages for better clarity.
- [fossology/fossology#3555: Fix pkgagent memory & encoding bugs](#) — Resolved unsafe `strcpy()`, NULL-before-free crashes, and HTML double-encoding in package info.
- [fossology/fossology#3554: Fix libfossrepo path string lengths](#) — Fixed `strlen()` misuse in tracking extension length during temporary path generation.
- [fossology/fossology#3513: Fix DAO SQL bugs & null safety](#) — Fixed SQL formatting bugs, array `intval()` misuse, and added full test coverage for License DAOs.
- [fossology/fossology#3477: Fix GroupController HTTP status codes](#) — Corrected null safety gaps and updated REST conventions (200 → 201/202) in member API endpoints.
- [fossology/fossology#3456: Fix N+1 queries & API crashes](#) — Eliminated redundant DB lookups and prevented `TypeError`s on synthetic license entries in the `/scanned` API.
- [fossology/fossology#3443: Update deprecated PHPUnit assertions](#) — Replaced deprecated `assertRegExp` calls across the test suite for modern PHPUnit compatibility.
- [fossology/fossology#3438: Fix API null safety & misleading errors](#) — Resolved unhandled null states, corrected status codes, and fixed typos in error payloads.

Software Developer Intern | Locara Labs

Aug. 2026 – Present

- Building production web and mobile applications for large-scale egocentric video datasets that power the next generation of robotics and embodied AI systems.
- Developing full-stack features and infrastructure across a modern stack including Next.js, React Native, Supabase, and Cloudflare.
- Building and supporting robust data pipelines and quality-assurance tooling to streamline high-throughput video dataset workflows for field teams and clients.
- Owning end-to-end feature delivery, participating in code reviews, and debugging production issues for internal users directly in production.

Software Engineer Intern | Tense AI

Dec. 2025 – Jun. 2026

- Architected scalable RESTful APIs and microservices using Node.js and Express, ensuring high performance and security for enterprise-grade applications.
- Built production-facing platform features across backend services, workflow automation, and AI-assisted product surfaces.
- Improved system scalability with queue-driven workflows and tighter coordination across product and engineering teams.
- Designed and shipped an AI-powered enterprise automation platform processing **500+ emails/day**, eliminating manual triage for **3 client teams**.
- Optimized end-to-end microservice latency by **40%** through async event-driven architecture and Redis caching across **6 containerized services**.

PROJECTS

Resync: UPI Autopay Recovery Platform

2026

Live at resync.biz | GitHub: github.com/mandar1045/Resync

Overview: Resync is the recovery layer for India's UPI Autopay ecosystem. Merchants register their existing autopay customers; Resync watches every expected charge and when one fails, it executes smart retries on the merchant's own payment gateway account. Pricing is success-based: 3% of recovered revenue. The platform supports both **direct recovery** (Resync charges on merchant's connected Razorpay/Cashfree keys) and **delegated recovery** (merchant charges on Resync's instruction, keeping money-moving secrets in their own infra).

Architecture & Technical Details:

- Architected a **9-microservice** platform in **Go** with **gRPC/Protobuf** service contracts:
 - **auth-svc** — Identity management, API key issuance (test/live keys), Supabase JWT verification
 - **sub-svc** — Subscription state machine (pending → active → past_due → cancelled), autopay lifecycle
 - **payment-svc** — Multi-gateway abstraction (Razorpay, Cashfree, PhonePe), idempotent charge execution, mandate management, AES-256-GCM credential encryption

- **dunning-svc** — Smart retry engine with UPI failure code classifier (30+ NPCI/Razorpay/Cashfree codes mapped to 8 internal categories), 4 retry strategies, delegated recovery with nudge sweeps
- **invoice-svc** — GST-compliant invoice generation, PDF rendering, S3/MinIO storage, platform fee collection via Razorpay Payment Links
- **webhook-svc** — Outbound event delivery with HMAC-SHA256 signature verification, retry with exponential backoff
- **checkout-svc** — Hosted checkout pages and recovery links for end-customer payment re-authorization
- **billing-cron** — Periodic billing sweep, autopay-missed detection, gateway mandate sync
- **api-gateway** — REST→gRPC translation, rate limiting, auth middleware, Supabase JWT and API key dual-auth
- Designed **UPI failure code classifier** that normalizes 30+ raw NPCI/gateway codes into 8 internal categories and routes to 4 context-aware retry strategies:
 - **Salary-day:** insufficient funds (U30, Z9) → retry on inferred pay-day (1st/15th IST) using historical payment observation streams
 - **Exponential backoff:** technical errors (U16, BT) → 2h, 4h, 8h doubling backoff
 - **Next-day:** limit exceeded (U68, Z8) → retry next day 10:30am IST
 - **Hard fail:** mandate revoked/expired (U14) → immediate subscription cancellation, no retry
- Orchestrated event-driven pipeline using **Apache Kafka** (via Redpanda) with **subscription_id** partition keys ensuring strict event ordering; combined with **Redis SETNX distributed locks** (TTL 30s) and structured idempotency keys (**charge:{sub_id}:{cycle}**) guaranteeing **zero double-charges** even under concurrent cron workers.
- Implemented **recovery attribution system** with 5 attribution types (**resync_charge**, **resync_settled**, **resync_delegated**, **resync_assisted**, **organic**) and a 72-hour attribution window, enabling accurate success-based billing (3% fee) while preventing unjustified charges for organic recoveries.
- Built **Node.js SDK** (@mandar1045/resync, npm-published) and **Python SDK** (**resync-sdk**, pip-installable) with full type hints, zero runtime dependencies, HMAC-SHA256 webhook verification, delegated recovery helpers, and comprehensive documentation.
- Developed **Next.js merchant dashboard** with real-time analytics (recovery rates, revenue recovered, failure code breakdowns), autopay management, gateway credential onboarding (Razorpay OAuth Partner flow), webhook configuration, and dunning policy customization.
- Deployed **full observability stack**: Prometheus metrics (per-service gRPC/HTTP latencies, failure code distributions, retry outcomes), Grafana dashboards, Alertmanager with email notifications via Resend, and structured logging with Go's **slog**.
- Containerized all 9 services + infrastructure via **Docker Compose** (dev) and **distroless Docker images** (~20MB each) for production. Infrastructure-as-code with **Terraform** targeting AWS ECS Fargate. Production deployed on Oracle Cloud Always-Free ARM VM with **Caddy** auto-HTTPS.
- Designed OpenAPI 3.0 specification (800+ lines) covering 15+ REST endpoints with comprehensive request/response schemas, enabling SDK auto-generation and API documentation.

Continuum: AI/ML Workflow Automation Platform

2026

Live at continuum.online

Overview: Production-grade AI/ML automation platform serving live enterprise clients, focused on intelligent email classification and workflow routing.

Technical Details:

- Designed ML-powered email classification engine supporting **15+ custom labels**, reducing manual triage time by **60%** for live enterprise clients using NLP pipelines with confidence-scored routing and configurable thresholds.
- Deployed containerized microservices on **VPS servers** with Docker and CI/CD automation, achieving **99.5% uptime** and sub-200ms API response times across production traffic.
- Implemented intelligent response generation using NLP models with contextual template selection, enabling automated replies for common query categories.
- Built admin dashboard with real-time classification accuracy monitoring, label management, and workflow configuration for non-technical users.

Real-Time Crowd Management System

2025

GitHub: github.com/mandar1045/Crowd-Management-System-software

Overview: Computer vision system for real-time crowd monitoring and stampede risk prediction, designed for large public venues and events.

Technical Details:

- Engineered multi-stream computer vision pipeline monitoring **4 simultaneous CCTV feeds** using YOLO-based object detection for crowd density estimation, achieving **91% detection accuracy** at 24 FPS.
- Implemented predictive stampede risk detection algorithm analyzing crowd flow patterns, density gradients, and movement velocity with automated alert dispatching, reducing detection-to-alert latency to **under 1.5 seconds**.
- Optimized inference throughput via GPU-accelerated frame batching and temporal skip-frame strategies, keeping **CPU overhead under 15%** on commodity hardware.
- Built real-time monitoring dashboard with live video feeds, density heat maps, zone-wise crowd counts, and historical trend visualization.

ADDITIONAL INFORMATION

Languages: English (Fluent), Hindi (Native), Marathi (Native)

Interests: Distributed Systems, Payment Infrastructure, Open-Source Software, Cloud-Native Architecture, Fintech

Location: Vellore, India (open to relocation)